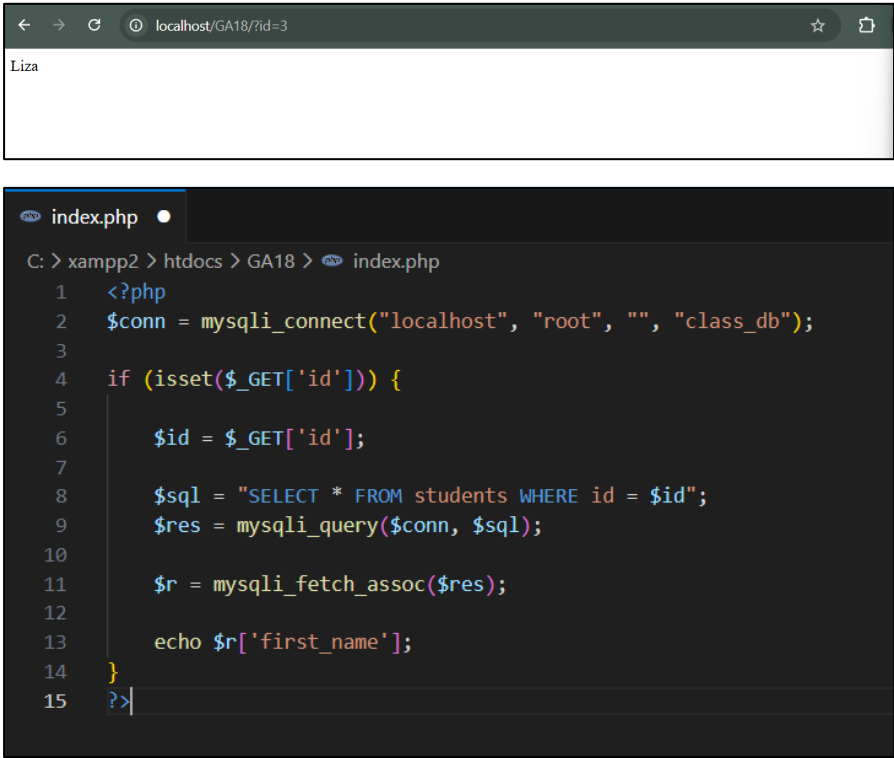
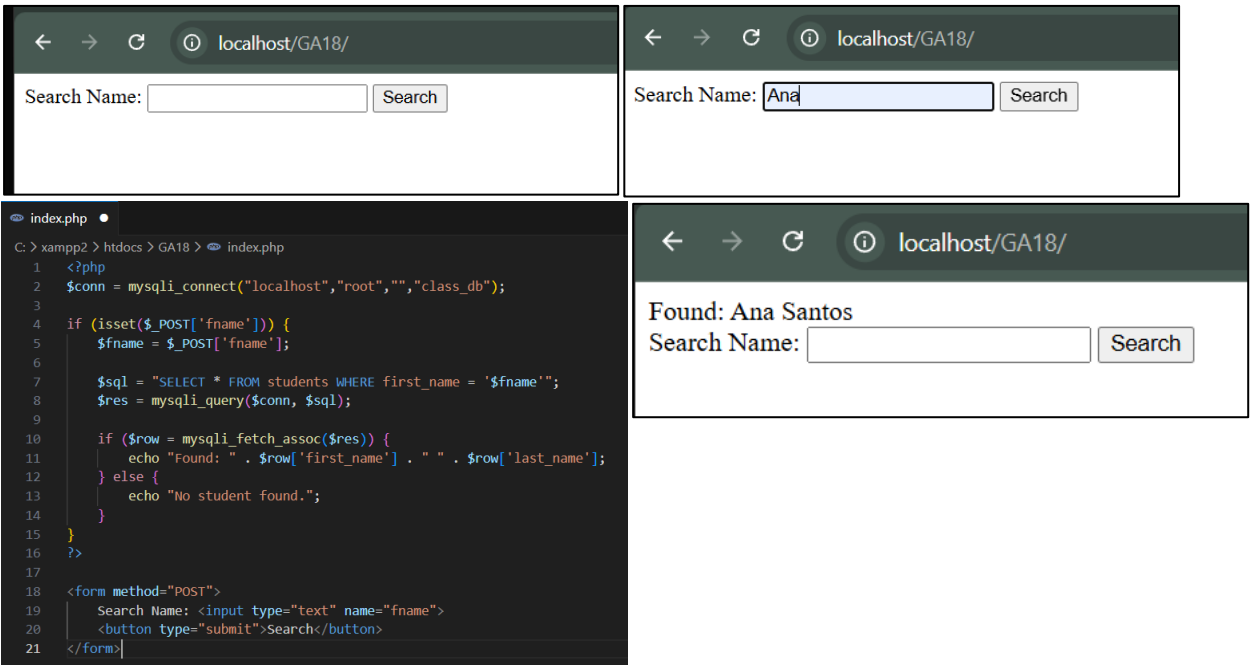


Scenario 1 — Using \$_POST instead of \$_GET



The original code failed because it tried to retrieve the ID using `$_POST`, which looks for **hidden form data** and **cannot see parameters passed in the URL**, leading to an "undefined index" error. To fix it, simply switch the variable to `$_GET` so it correctly **reads data from the URL address bar**, and wrap the code in an `isset()` check to ensure the database query only runs when an ID is actually present.

Scenario 2 — Missing quotes in SQL when using POST



The error happens because the script **tries to grab the fname** value **the moment the page loads**, before you've even had a chance to submit the form. Since the form hasn't been sent yet, that piece of data doesn't exist, triggering the warning. To resolve this, you need to wrap the database code inside an if (`isset(...)`) block. This tells PHP to wait and only attempt to read the variable **after the submit button has been clicked**.

Scenario 3 — SQL injection vulnerability

localhost/GA18/?age=22

Miguel

index.php

```
C: > xampp2 > htdocs > GA18 > index.php
1 <?php
2 $conn = mysqli_connect("localhost", "root", "", "class_db");
3
4 if (isset($_GET['age'])) {
5     $age = $_GET['age'];
6
7     $stmt = mysqli_prepare($conn, "SELECT * FROM students WHERE age = ?");
8
9     mysqli_stmt_bind_param($stmt, "i", $age);
10
11     mysqli_stmt_execute($stmt);
12
13     $res = mysqli_stmt_get_result($stmt);
14
15     while ($row = mysqli_fetch_assoc($res)) {
16         echo $row['first_name'] . "<br>";
17     }
18 }
19 ?>
```

The vulnerability exists because the original code directly pastes raw user input into the SQL command. This **allows an attacker to "inject"** their own SQL logic (like **1 OR 1=1**), tricking the database into revealing all records instead of just one. To secure it, you must use a **Prepared Statement**. This technique **separates the code from the data** by using a placeholder (a **?**). The database then treats the user's input strictly as a value, never as a command.

Scenario 4 — Forgetting to validate empty POST field

localhost/GA18/

First Name:
Last Name:
Email:
Add Student

localhost/GA18/

Error: Please fill in all fields!
First Name:
Last Name:
Email:
Add Student

localhost/GA18/

Error: Please fill in all fields!
First Name: Lucio
Last Name: Malpoy
Email: wizardElf@harrypotter.com
Add Student

localhost/GA18/

Success: Student inserted!
First Name:
Last Name:
Email:
Add Student

localhost/phpmyadmin/index.php?route=/sql&db=class_db&table=students

phpMyAdmin

Showing rows 0 - 5 (6 total, Query took 0.0003 seconds)
SELECT * FROM `students`
Number of rows: 25
Filter rows: Search this table
Sort by key: N

	id	first_name	last_name	email	age
☐	1	Ana	Santos	ana@example.com	20
☐	2	Miguel	Reyes	miguel@example.com	22
☐	3	Liza	Cruz	liza@example.com	19
☐	4	John	Tomes	john@example.com	21
☐	5	Maria	Velasco	maria@example.com	23
☐	13	Lucio	Malpoy	wizardElf@harrypotter.com	N/A

index.php

```
C: > xampp2 > htdocs > GA18 > index.php
1 <?php
2 $conn = mysqli_connect("localhost", "root", "", "class_db");
3
4 if (isset($_POST['submit'])) {
5     $first = $_POST['fname'];
6     $last = $_POST['lname'];
7     $email = $_POST['email']; // 1. Get the email
8
9     // 2. Validate all three fields
10    if (!empty(trim($first)) && !empty(trim($last)) && !empty(trim($email))) {
11
12        // 3. Update Query to include email
13        $sql = "INSERT INTO students (first_name, last_name, email)
14              VALUES ('$first', '$last', '$email')";
15
16        // We use a try-catch block to handle the duplicate error gracefully
17        try {
18            if (mysqli_query($conn, $sql)) {
19                echo "Success: Student inserted!";
20            }
21        } catch (mysqli_sql_exception $e) {
22            echo "Error: That email is already taken!";
23        }
24    } else {
25        echo "Error: Please fill in all fields!";
26    }
27 }
28
29 ?>
30
31 <form method="POST">
32     First Name: <input type="text" name="fname"><br>
33     Last Name: <input type="text" name="lname"><br>
34     Email: <input type="email" name="email"><br>
35     <button type="submit" name="submit">Add Student</button>
36 </form>
```

The original code failed because it **blindly accepts** whatever input it gets (even if it's blank) and tries to save it. If a user clicks "Submit" without typing anything, the database creates a "ghost" row with no name. To fix it, we add a **validation step** using **!empty()** to ensure the user actually typed some text before running the database command.

Scenario 5 — Wrong key name in POST

localhost/GA18/
Enter Email: Search

localhost/GA18/
Enter Email: miguel@example.com Search

localhost/GA18/
Found: Miguel
Enter Email: Search

The original code failed because of a **spelling error**: it tried to retrieve data using the key 'emial', which does not match the input name defined in the HTML form (name="email"), causing PHP to return an "undefined index" error. To fix it, simply **correct the spelling to \$_POST['email']** so that the PHP variable name matches the HTML form name exactly.

index.php X

C: > xampp2 > htdocs > GA18 > index.php
1 <?php
2 \$conn = mysqli_connect("localhost", "root", "", "class_db");
3
4 if (isset(\$_POST['email'])) {
5
6 \$email = \$_POST['email'];
7
8 \$sql = "SELECT * FROM students WHERE email='\$email'";
9 \$res = mysqli_query(\$conn, \$sql);
10
11 if (\$row = mysqli_fetch_assoc(\$res)) {
12 echo "Found: " . \$row['first_name'];
13 } else {
14 echo "No email found.";
15 }
16 }
17 ?>
18
19 <form method="POST">
20 Enter Email: <input type="text" name="email">
21 <button type="submit">Search</button>
22 </form>

Scenario 6 — Unsafe direct use of GET in DELETE

myadmin/index.php?route=/sql&db=class_db&table=students
Server: 127.0.0.1 > Database: class_db > Table: students
Showing rows 0 - 5 (6 total, Query took 0.0003 seconds.)
SELECT * FROM `students`
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code]
Show all | Number of rows: 25 | Filter rows: Search this table | Sort
Extra options
id first_name last_name email
1 Ana Santos ana@example.com
2 Miguel Reyes miguel@example.com
3 Liza Cruz liza@example.com
4 John Torres john@example.com
5 Maria Velasco maria@example.com
13 Lucio Malpoy blondewizardElif@harrypotter

localhost/GA18/?id=13
Student deleted successfully.

myadmin/index.php?route=/sql&db=class_db&table=students
Server: 127.0.0.1 > Database: class_db > Table: students
Showing rows 0 - 4 (5 total, Query took 0.0004 seconds.)
SELECT * FROM `students`
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code]
Show all | Number of rows: 25 | Filter rows: Search this table | Sort
Extra options
id first_name last_name email age
1 Ana Santos ana@example.com 20
2 Miguel Reyes miguel@example.com 22
3 Liza Cruz liza@example.com 19
4 John Torres john@example.com 21
5 Maria Velasco maria@example.com 23

By directly pasting the URL input into the delete command, a hacker can trick the database into deleting **every student at once** (e.g., by entering ?id=2 OR 1=1, which makes the condition "true" for every row). To fix it, simply use a **Prepared Statement to strictly treat the ID as a value**, or use intval() to force the input to be an integer, ensuring no malicious code can be executed.

index.php X

C: > xampp2 > htdocs > GA18 > index.php
1 <?php
2 \$conn = mysqli_connect("localhost", "root", "", "class_db");
3
4 if (isset(\$_GET['id'])) {
5
6 \$id = \$_GET['id'];
7
8 \$stmt = mysqli_prepare(\$conn, "DELETE FROM students WHERE id = ?");
9
10 mysqli_stmt_bind_param(\$stmt, "i", \$id);
11
12 if (mysqli_stmt_execute(\$stmt)) {
13 echo "Student deleted successfully.";
14 } else {
15 echo "Error deleting student.";
16 }
17 } else {
18 echo "No ID provided to delete.";
19 }
20
21 ?>

Scenario 7 — Query fails but script continues

localhost/GA18/

ID to Update:

New Email:

Update

localhost/GA18/

ID to Update: 5

New Email: mariavelasco5@gmail.com

Update

	id	first_name	last_name	email	age
☐ Edit ☐ Copy ☐ Delete	1	Ana	Santos	ana@example.com	20
☐ Edit ☐ Copy ☐ Delete	2	Miguel	Reyes	miguel@example.com	22
☐ Edit ☐ Copy ☐ Delete	3	Liza	Cruz	liza@example.com	19
☐ Edit ☐ Copy ☐ Delete	4	John	Torres	john@example.com	21
☐ Edit ☐ Copy ☐ Delete	5	Maria	Velasco	mariavelasco5@gmail.com	23

	id	first_name	last_name	email	age
☐ Edit ☐ Copy ☐ Delete	1	Ana	Santos	ana@example.com	20
☐ Edit ☐ Copy ☐ Delete	2	Miguel	Reyes	miguel@example.com	22
☐ Edit ☐ Copy ☐ Delete	3	Liza	Cruz	liza@example.com	19
☐ Edit ☐ Copy ☐ Delete	4	John	Torres	john@example.com	21
☐ Edit ☐ Copy ☐ Delete	5	Maria	Velasco	mariavelasco5@gmail.com	23

The original code failed because it **lacked quotes around the email address** (causing an SQL syntax error) and it unconditionally printed "Updated!" even if the database command failed, misleading the user. To fix it, simply **add single quotes** around the `$email` variable in the query, and **wrap the echo statement** in an if check to confirm the query returned true before claiming success.

index.php

```
C: > xampp2 > htdocs > GA18 > index.php
1 <?php
2 $conn = mysqli_connect("localhost", "root", "", "class_db");
3
4 if (isset($_POST['id']) && isset($_POST['email'])) {
5
6     $id = $_POST['id'];
7     $email = $_POST['email'];
8
9     $sql = "UPDATE students SET email='$email' WHERE id=$id";
10
11     if (mysqli_query($conn, $sql)) {
12         echo "Updated successfully!";
13     } else {
14         echo "Error: " . mysqli_error($conn);
15     }
16 }
17 ?>
18
19 <form method="POST">
20 ID to Update: <input type="text" name="id"><br>
21 New Email: <input type="text" name="email"><br>
22 <button type="submit">Update</button>
23 </form>
```

Scenario 8 — Missing mysqli_fetch_assoc loop

localhost/GA18/

ana@example.com
miguel@example.com
liza@example.com
john@example.com
mariavelasco5@gmail.com

index.php

```
C: > xampp2 > htdocs > GA18 > index.php
1 <?php
2 $conn = mysqli_connect("localhost", "root", "", "class_db");
3 $res = mysqli_query($conn, "SELECT * FROM students");
4
5 while ($row = mysqli_fetch_assoc($res)) {
6     echo $row['email'] . "<br>";
7 }
8 ?>
```

The original code failed because `mysqli_fetch_assoc()` retrieves only **one row at a time** from the database. Since the function was called just once, the script fetched the first student, printed their email, and then stopped, ignoring the rest of the list. To fix it, simply **wrap the fetch command inside a while loop**. This tells PHP to keep fetching the "next" row over and over again until there are no more rows left.

Scenario 9 — Using GET but link sends POST

localhost/GA18/

Class List:

[View Ana \(ID 1\)](#)

[View Miguel \(ID 2\)](#)

[View Liza \(ID 3\)](#)

[View John \(ID 4\)](#)

[View Maria \(ID 5\)](#)

localhost/GA18/index.php?id=4

Student Profile

Name: John Torres

Email: john@example.com

Age: 21

Class List:

[View Ana \(ID 1\)](#)

[View Miguel \(ID 2\)](#)

[View Liza \(ID 3\)](#)

[View John \(ID 4\)](#)

[View Maria \(ID 5\)](#)

index.php

```
C: > xampp2 > htdocs > GA18 > index.php
4 if (isset($_GET['id'])) {
5     $id = $_GET['id'];
6
7     $sql = "SELECT * FROM students WHERE id = $id";
8     $res = mysqli_query($conn, $sql);
9     $student = mysqli_fetch_assoc($res);
10
11     if ($student) {
12         echo "<div>Student Profile</div>";
13         echo "Name: " . $student['first_name'] . " - " . $student['last_name'] . "<br>";
14         echo "Email: " . $student['email'] . "<br>";
15         echo "Age: " . $student['age'];
16     } else {
17         echo "Student not found.";
18     }
19 }
20 ?>
21
22 <hr>
23
24 <div>Class List:</div>
25 <div>
26 <a href="index.php?id=1">View Ana (ID 1)</a></div>
27 <a href="index.php?id=2">View Miguel (ID 2)</a></div>
28 <a href="index.php?id=3">View Liza (ID 3)</a></div>
29 <a href="index.php?id=4">View John (ID 4)</a></div>
30 <a href="index.php?id=5">View Maria (ID 5)</a></div>
31 </div>
```

The original code failed because it uses `$_POST` to try and retrieve the ID, but standard HTML hyperlinks (`<a href="...">`) always send data as a **GET** request (visible in the URL). Since **links cannot send POST data**, the code couldn't find the ID. To fix it, simply switch the variable to `$_GET['id']` to correctly read the parameter from the URL link.

Scenario 10 — Wrong variable used in SQL

localhost/GA18/

Enter Age: 19

Search

Found student with age 19: Liza

Enter Age:

Search

The original code failed because of a **mismatch in variable names**: the script defined \$age, but the SQL command attempted to use \$aeg. PHP treats these as distinct variables, and since \$aeg was never assigned a value, it triggered an "Undefined variable" error. To fix it, simply **correct the spelling** in the SQL query to \$age so it matches the variable defined at the start of the script.

index.php

```
1 <?php
2 $conn = mysqli_connect("localhost", "root", "", "class_db");
3
4 if (isset($_POST['age'])) {
5
6     $age = $_POST['age'];
7
8     $sql = "SELECT * FROM students WHERE age = $age";
9     $res = mysqli_query($conn, $sql);
10
11     if ($row = mysqli_fetch_assoc($res)) {
12         echo "Found student with age " . $age . ": " . $row['first_name'];
13     } else {
14         echo "No student found with that age.";
15     }
16 }
17 ?>
18
19 <form method="POST">
20     Enter Age: <input type="text" name="age">
21     <button type="submit">Search</button>
22 </form>
```

Scenario 11 — Mismatched method (expects POST but form sends GET)

localhost/GA18/

Email: liza@example.com

Save

Saving email: liza@example.com

The original code failed because of a **mismatch in HTTP methods**: the HTML form transmits data via GET (putting it in the URL), but the PHP script attempts to retrieve it using \$_POST. Because the data was sent to the wrong "channel," PHP could not find the "email" index. To fix it, simply update the HTML form attribute to **method="POST"**. This aligns the form with the PHP script and is the standard practice for actions that "save" data.

index.php

```
1 <form method="POST" action="index.php">
2     Email: <input type="text" name="email">
3     <button type="submit">Save</button>
4 </form>
5
6 <?php
7 if (isset($_POST['email'])) {
8     $email = $_POST['email'];
9     echo "Saving email: " . $email;
10 }
11 ?>
```

Scenario 12 — Numeric GET used inside quotes

localhost/GA18/index.php?id=4

Student Found

The original code is inefficient because it wraps the numeric ID in quotes, **treating it as a text string** (e.g., '3'). This forces the database to perform "implicit type conversion" on every row to match the data, which often prevents it from using high-speed indexes and slows down the query. To fix it, simply **remove the single quotes** around the variable. It is also best practice to use intval() to strictly force the input to be a number, which adds a layer of security against SQL injection.

index.php

```
1 <?php
2 $conn = mysqli_connect("localhost", "root", "", "class_db");
3
4 if (isset($_GET['id'])) {
5
6     $id = intval($_GET['id']);
7     $sql = "SELECT * FROM students WHERE id = $id";
8     $res = mysqli_query($conn, $sql);
9
10     if ($row = mysqli_fetch_assoc($res)) {
11         echo "<h1>Student Found</h1>";
12         echo "Name: " . $row['first_name'] . " " . $row['last_name'];
13     } else {
14         echo "No student found with ID " . $id;
15     }
16 } else {
17     echo "Please add ?id=1 to the URL to test.";
18 }
19 ?>
```

Scenario 13 — Missing WHERE clause in UPDATE

	id	first_name	last_name	email	age
☐ Edit ☐ Copy ☐ Delete	1	Ana	Santos	aname@mail.com	20
☐ Edit ☐ Copy ☐ Delete	2	Miguel	Reyes	miguel@example.com	22
☐ Edit ☐ Copy ☐ Delete	3	Liza	Cruz	liza@example.com	19
☐ Edit ☐ Copy ☐ Delete	4	John	Torres	johnpogi@email.com	21
☐ Edit ☐ Copy ☐ Delete	5	Maria	Velasco	mariavelasco5@gmail.com	23

Update Student Email

Student ID:

New Email:

Update Email

Success! Student ID 3 was updated.

Update Student Email

Student ID:

New Email:

Update Email

The original code failed because it is missing a filter condition. **Without a WHERE clause**, the UPDATE command applies the change to **every single row** in the table, effectively overwriting the email address for every student in your database with the same value. To fix it, simply **add a WHERE clause** that specifies the unique ID of the student you want to update.

```
index.php
C:\xampp2\htdocs> GA18 > index.php
1 <?php
2 $conn = mysqli_connect("localhost", "root", "", "class_db");
3
4 if (isset($_POST['email']) && isset($_POST['id'])) {
5
6     $newEmail = $_POST['email'];
7     $id = $_POST['id'];
8
9     $sql = "UPDATE students SET email='$newEmail' WHERE id=$id";
10
11     // Run the query
12     if (mysqli_query($conn, $sql)) {
13
14         // CHECK: Did we actually change anything?
15         if (mysqli_affected_rows($conn) > 0) {
16             echo "<h3>✔ Success! Student ID $id was updated.</h3>";
17         } else {
18             echo "<h3>⚠ Query ran, but nothing changed.</h3>";
19             echo "Possible reasons:<br>";
20             echo "1. Student ID $id does not exist.<br>";
21             echo "2. The email was already '$newEmail' (no change needed).";
22         }
23     } else {
24         echo "SQL Error: " . mysqli_error($conn);
25     }
26 }
27
28 ?>
29
30 <hr>
31 <h3>Update Student Email</h3>
32 <form method="POST">
33     Student ID: <input type="number" name="id" required><br><br>
34     New Email: <input type="email" name="email" required><br><br>
35     <button type="submit">Update Email</button>
36 </form>
```

Scenario 14 — Using POST array incorrectly

First Name:

Last Name:

Email:

Add Student

Student inserted successfully!

First Name:

Last Name:

Email:

Add Student

	id	first_name	last_name	email	age
☐ Edit ☐ Copy ☐ Delete	1	Ana	Santos	aname@mail.com	20
☐ Edit ☐ Copy ☐ Delete	2	Miguel	Reyes	miguel@example.com	22
☐ Edit ☐ Copy ☐ Delete	3	Liza	Cruz	lizasumbbrero@email.com	19
☐ Edit ☐ Copy ☐ Delete	4	John	Torres	johnpogi@email.com	21
☐ Edit ☐ Copy ☐ Delete	5	Maria	Velasco	mariavelasco5@gmail.com	23
☐ Edit ☐ Copy ☐ Delete	14	Luigi	Fetuso	luigi@example.com	NULL

```
index.php
C:\xampp2\htdocs> GA18 > index.php
1 <?php
2 $conn = mysqli_connect("localhost", "root", "", "class_db");
3
4 if (isset($_POST['first_name'])) {
5
6     $first = $_POST['first_name'];
7     $last = $_POST['last_name'];
8     $email = $_POST['email'];
9
10     $sql = "INSERT INTO students (first_name, last_name, email)
11         VALUES ('$first', '$last', '$email')";
12
13     if (mysqli_query($conn, $sql)) {
14         echo "Student inserted successfully!";
15     } else {
16         echo "Error: " . mysqli_error($conn);
17     }
18 }
19
20
21 <form method="POST">
22     First Name: <input type="text" name="first_name"><br>
23     Last Name: <input type="text" name="last_name"><br>
24     Email: <input type="text" name="email"><br>
25     <button type="submit">Add Student</button>
26 </form>
```

The original code failed because it didn't wrap the text values (names, email) in **single quotes**, so the database tried to read them as column names instead of data. It also uses array syntax like `$data[first_name]` **directly inside a double-quoted string** is risky and often fails. It's safer to use curly braces `{ $data['first_name'] }` or extract them to variables first. To fix it, simply **extract the data into specific variables first** (which fixes the array syntax), and then wrap those variables in **single quotes** inside the SQL command.

Scenario 15 — GET parameter used inside SQL without sanitization

← → ↻ ⓘ localhost/GA18/

Showing Page 1 of 2

(Total Students: 6)

ID: 1 - Ana
ID: 2 - Miguel
ID: 3 - Liza
ID: 4 - John
ID: 5 - Maria

[Next »](#)

← → ↻ ⓘ localhost/GA18/?page=2

Showing Page 2 of 2

(Total Students: 6)

ID: 14 - Luigi

[« Previous](#)

The original code failed because it **never checked** the **total number of students**. It calculated the math blindly, allowing you to go to "Page 100" even if you only have 10 students. To fix it, we need to run a **Count Query** first to calculate the maximum number of pages (\$totalPages). Then, add logic to ensure the current page never goes below 1 or above that maximum.

index.php

```
C:\xampp2\htdocs> GA18 > index.php
1  <?php
2  $conn = mysqli_connect("localhost", "root", "", "class_db");
3  $limit = 5;
4
5  $count_sql = "SELECT COUNT(*) as total FROM students";
6  $count_res = mysqli_query($conn, $count_sql);
7  $count_row = mysqli_fetch_assoc($count_res);
8  $total_students = $count_row['total'];
9
10 $total_pages = ceil($total_students / $limit);
11
12 if ($total_pages == 0) { $total_pages = 1; }
13
14 if (isset($_GET['page'])) {
15     $page = intval($_GET['page']);
16 } else {
17     $page = 1;
18 }
19
20 if ($page < 1) {
21     $page = 1;
22 }
23
24 if ($page > $total_pages) {
25     $page = $total_pages;
26 }
27
28 $offset = ($page - 1) * $limit;
29 $sql = "SELECT * FROM students LIMIT $offset, $limit";
30 $res = mysqli_query($conn, $sql);
31
32 echo "<h3>Showing Page $page of $total_pages</h3>";
33 echo "(Total Students: $total_students)<br><hr>";
34
35 while ($row = mysqli_fetch_assoc($res)) {
36     echo "ID: " . $row['id'] . " - " . $row['first_name'] . "<br>";
37 }
38
39 echo "<hr>";
40
41 if ($page > 1) {
42     $prev = $page - 1;
43     echo "<a href='?page=$prev'>&laquo; Previous</a> ";
44 }
45
46 if ($page < $total_pages) {
47     $next = $page + 1;
48     echo " <a href='?page=$next'>Next &raquo;</a>";
49 }
50 >>
```